

Video Processing — Complete Algorithm Reference

Course 0512-4263 (Prof. Shai Avidan, Tel-Aviv University)

Compiled from the course html-book, chapters 1–12

Scope. This is a reference to *every* algorithm/method taught in the course (chapters 1–12), each in a fixed six-part template: *Purpose* → *When to use* → *When to avoid* → *The algorithm (pseudocode + math)* → *Parameters* → *Short explanation*. Sections follow the course chapter order. Every formula is grounded in the source; items not present are marked **[not specified in source]**.

Contents

1	Motion Representation (Chapter 1)	3
1.1	Linear Transformation (2×2)	3
1.2	Affine Transformation (2×3)	3
1.3	Homography (3×3)	4
1.4	Projection Matrix (pinhole camera)	4
2	Optical Flow (Chapter 2)	6
2.1	Lucas–Kanade	6
2.2	Harris Corner Detector	6
2.3	Horn–Schunck	7
3	Applications of Motion Estimation (Chapter 3)	9
3.1	Super-Resolution (ML / MAP)	9
3.2	BRIEF Descriptor	9
3.3	RANSAC	10
3.4	Hyperlapse (cost + dynamic programming)	10
4	Online Tracking: Kalman & Particle Filters (Chapter 4)	12
4.1	Bayesian Recursive Tracking (derivation)	12
4.2	Kalman Filter	12
4.3	Particle Filter (Condensation)	13
5	Dynamic Texture Synthesis (Chapter 5)	15
5.1	SVD + ARMA State-Space Texture Synthesis	15
6	Layer Representation (Chapter 6)	16
6.1	Layer Decomposition (LK + K-means) and Alpha Compositing	16
6.2	K-means Clustering	16
7	Video Magnification (Chapter 7)	18
7.1	Eulerian Video Magnification	18
8	Background Subtraction (Chapter 8)	19
8.1	Per-pixel Gaussian Mixture Model (GMM)	19
9	Video Colorization & Geodesic Distance (Chapter 9)	20

9.1 Geodesic Distance Colorization	20
9.2 Dijkstra's Algorithm	20
9.3 Fast Sweeping (Danielsson)	21
10 Video Matting (Chapter 10)	22
10.1 Trimap Estimation (KDE + geodesic)	22
10.2 Opacity Map (trimap refinement)	22
10.3 Matting (pure-foreground recovery)	23
10.4 Fast KDE via Low-Rank Kernel	23
11 Re-targeting & PatchMatch (Chapter 11)	24
11.1 Bidirectional-Similarity Re-targeting	24
11.2 PatchMatch	24
11.3 Applications: Inpainting, Reshuffling, Object Removal	25
12 Intrinsic Images from Sequences (Chapter 12)	26
12.1 Weiss Intrinsic Images (median of derivative filters)	26
Course-wide Pattern	27
Parameter Index and Glossary	28

1 Motion Representation (Chapter 1)

Every motion a camera perceives is a matrix acting on coordinates. The models form a ladder of increasing expressiveness (and estimation difficulty): linear $\rightarrow$ affine $\rightarrow$ homography $\rightarrow$ full 3D projection. *Counting rule:* a model with k free parameters needs at least $\lceil k/2 \rceil$ point pairs (each pair gives 2 equations); with more pairs the system is over-determined and solved by Least Squares $x = (A^T A)^{-1} A^T b$.

1.1 Linear Transformation (2×2)

Purpose. Represent rotation, scale, and shear of image coordinates by a 2×2 matrix, $p' = Ap$.

When to use. Small/far objects whose motion has no translation or perspective; the cheapest model (4 params).

When to avoid. Whenever translation is needed — a pure 2×2 matrix always fixes the origin ($A(0,0)^T = (0,0)^T$); use affine instead.

The algorithm.

$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$. Stack point pairs into $\tilde{A}\tilde{x} = b$ with $\tilde{x} = (a \ b \ c \ d)^T$; each pair gives 2 rows:

$$\begin{pmatrix} x & y & 0 & 0 \\ 0 & 0 & x & y \\ \vdots & & & \end{pmatrix} \begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix} = \begin{pmatrix} x' \\ y' \\ \vdots \end{pmatrix}.$$

Solve exactly with 2 pairs (invert 4×4) or over-determined with LS.

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
a, b, c, d	Rotation/scale/shear jointly	More free params = more flexible but more noise-sensitive
# point pairs	Determinacy of the fit	≥ 2 needed; more pairs $\Rightarrow$ LS averages out noise

Short explanation. “Multiply by a 2×2 matrix.” Its four numbers encode rotation, scaling and shear together, but it can never shift the image because the origin is a fixed point. Estimate it from before/after point pairs (2 equations each); with extra pairs the system is over-determined and Least Squares gives the best average fit.

1.2 Affine Transformation (2×3)

Purpose. Linear transform *plus* translation, using homogeneous coordinates; preserves parallelism.

When to use. Medium/closer objects; layer motion (Ch. 6 clusters affine motions). 6 params, keeps parallel lines parallel.

When to avoid. When true perspective is present (converging parallels) — use homography; or when 6 params overfit a tiny target (use translation/similarity).

The algorithm.

Append a homogeneous “1”: $\begin{pmatrix} x' \\ y' \end{pmatrix} = (A \ t) \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & c \\ d & e & f \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$. Each pair gives 2 rows in 6 unknowns

$\Rightarrow$ need ≥ 3 pairs; solve $\tilde{A}\tilde{x} = b$ by LS.

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
A (4 params)	Rotation/scale/shear	As linear model
$t = (t_x, t_y)$	Translation	Shifts the origin anywhere (the homogeneous “1”)
# point pairs	Determinacy	≥ 3 pairs (= 6 equations)

Short explanation. The homogeneous “1” gives the matrix a slot to add a constant, so it can translate as if by multiplication. Affine does rotation + scale + shear + translation and always keeps parallel lines parallel (a square becomes a parallelogram, never a trapezoid). It needs 3 point pairs.

1.3 Homography (3×3)

Purpose. Model a planar perspective mapping (a flat world plane $\rightarrow$ image plane), up to scale.

When to use. Panorama stitching, document “flattening”, AR markers, mapping a tilted plane to top-down. Large/close flat objects.

When to avoid. Non-planar / full-3D scenes (needs the projection matrix); estimation is noise-sensitive (8 params).

The algorithm.

$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} \cong H \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$, where $\cong$ is equality *up to scale* (divide by the third coordinate: $x' = H_1p/H_3p$, $y' = H_2p/H_3p$). Rearranging gives a homogeneous system $Ax = 0$ (with A of size $2N \times 9$); 4 point pairs give 8×9 , and a normalisation constraint on the last row fixes the overall scale.

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
H (9 entries, 8 free)	Perspective warp	Overall scale is free (1 redundancy) $\Rightarrow$ normalise
# point pairs	Determinacy	≥ 4 pairs (8 equations)
Normalisation	Numerical stability	Constrains scale; avoids huge values

Short explanation. Homography is affine plus true perspective: parallel lines may converge (train tracks meeting). The 3×3 matrix is meaningful only after dividing by the last coordinate — that division creates the perspective. Only 8 of its 9 numbers are independent, so 4 point pairs suffice.

1.4 Projection Matrix (pinhole camera)

Purpose. Map a 3D world point (X, Y, Z) to a 2D pixel (u, v) through the pinhole-camera pipeline.

When to use. Relating world geometry to pixels: robotics, AR, self-driving, SLAM, photogrammetry.

When to avoid. When the lens distortion matters (undistort first) or intrinsics are uncalibrated.

The algorithm.

Three composed stages (world $\rightarrow$ camera $\rightarrow$ image $\rightarrow$ pixels):

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} \cong \underbrace{\begin{pmatrix} k_u & 0 & u_0 \\ 0 & k_v & v_0 \\ 0 & 0 & 1 \end{pmatrix}}_{\text{intrinsics (pixels)}} \underbrace{\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1/f & 0 \end{pmatrix}}_{\text{pinhole (1/Z)}} \underbrace{\begin{pmatrix} R_{3 \times 3} & t_{3 \times 1} \\ 0 & 1 \end{pmatrix}}_{\text{extrinsics } R, t} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix} = P \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}.$$

Perspective comes from triangle similarity $x = Xf/Z$, $y = Yf/Z$ (far = small). P is 3×4 (11 free params: 6 extrinsic + 5 intrinsic).

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
R, t (extrinsics)	Camera pose (where it is/points)	Change when the camera moves
f, k_u, k_v, u_0, v_0 (intrinsic)	Lens + sensor mapping	Calibrated once; f sets zoom, u_0, v_0 principal point
Depth Z	Projected size ($1/Z$)	Larger Z : object projects smaller

Short explanation. A world point is rotated/translated into camera coordinates (R, t) , projected onto the image plane by pinhole similarity (the $1/Z$ that shrinks distant things), then converted from cm to pixels (k_u, k_v, u_0, v_0) . Multiplying the three matrices gives one 3×4 projection matrix P that turns any $(X, Y, Z, 1)$ into a pixel. Extrinsics say where the camera is; intrinsics say how it maps rays to pixels.

2 Optical Flow (Chapter 2)

Optical flow asks, for every pixel, where it moved in the next frame: find (u, v) with $I_1(x+u, y+v) = I_2(x', y')$. It rests on the *constant brightness constraint* (a moving point keeps its intensity), which breaks under specularities, shadows, and auto-exposure.

2.1 Lucas–Kanade

Purpose. Recover local motion (u, v) (or a warp $W(X; P)$) by assuming a whole window moves together and solving a tiny least-squares system from a first-order Taylor expansion.

When to use. Small motions, textured/corner patches, high frame rate; tracking, stabilisation, SLAM — sparse flow where speed matters (a 2×2 inverse per window).

When to avoid. Large jumps; flat/edge-only patches (aperture problem, $A^T A$ singular); specular reflections; when dense per-pixel flow is needed (use Horn–Schunck).

The algorithm.

Brightness constancy + Taylor gives $I_x u + I_y v + I_t = 0$ per pixel. Stack the window into $Ax = b$ and minimise $E = \|Ax - b\|^2$:

$$x = (A^T A)^{-1} A^T b, \quad A^T A = \begin{pmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{pmatrix}.$$

General warp form: $P^* = \arg \min_P \sum_X [I_2(W(X; P)) - I_1(X)]^2$, solved incrementally with $\Delta P^* = -(B^T B)^{-1} B^T I_t$, $B = \nabla I_2 \partial W / \partial P$ (for affine $\partial W / \partial P = \begin{pmatrix} x & y & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x & y & 1 \end{pmatrix}$).

Algorithm 1: Incremental Lucas–Kanade

Input: I_1, I_2 ; window W ; warp model $W(X; P)$; tolerance ϵ

Output: Motion (u, v) / warp params P

```

1 Initialise  $P$  (identity / zero);
2 repeat
3   Warp  $I_2$  by  $P$ ;  $I_t = I_2(W(X; P)) - I_1(X)$ ;
4    $B = \nabla I_2 \partial W / \partial P$  over the window;
5    $\Delta P^* = -(B^T B)^{-1} B^T I_t$ ;  $P \leftarrow P + \Delta P^*$ ;
6 until  $\|\Delta P\| < \epsilon$ ;
```

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
Window size n	# equations per solve	Larger: robust but mixes motions; smaller: worse aperture problem
Warp model	Translation (2) vs. affine (6)	Richer fits larger windows, needs more data
Pyramid levels	Max recoverable motion	More levels handle bigger motion, coarse-to-fine

Short explanation. Instead of searching for matching pixels, LK uses calculus: intensity change $\approx$ gradient $\cdot$ motion. One pixel is one equation but there are two unknowns, so LK assumes a window shares one motion, giving an over-determined system solved by a 2×2 inverse (fast). It fails on flat/edge patches (rank-deficient) and large motion, which an image pyramid fixes.

2.2 Harris Corner Detector

Purpose. Find distinctive “interest points” (corners) — exactly the pixels where LK is rank-2 and trackable.

When to use. As the front-end of tracking: detect corners on frame 1, track them with LK (SLAM, 3D reconstruction).

When to avoid. Textureless regions; when dense flow is needed; scale-sensitive (combine with a pyramid).

The algorithm.

The self-difference $E(u, v) = \sum_W [I(x+u, y+v) - I(x, y)]^2$ linearises to the quadratic form

$$E(u, v) = (u \ v) H \begin{pmatrix} u \\ v \end{pmatrix}, \quad H = \sum_W \begin{pmatrix} I_x^2 & I_x I_y \\ I_y I_x & I_y^2 \end{pmatrix} (= A^T A),$$

with eigenvalues $\lambda_{\min} \leq \lambda_{\max}$: rank 0 flat, 1 edge, 2 corner.

Algorithm 2: Harris / structure-tensor detection

Input: Image I ; window g ; sensitivity α ; threshold θ

Output: Corner keypoints

- 1 $(I_x, I_y) = \text{grad}(I)$; $I_{xx} = I_x \cdot I_x$, $I_{xy} = I_x \cdot I_y$, $I_{yy} = I_y \cdot I_y$;
- 2 $S_{xx} = \text{conv}(I_{xx}, g)$, etc. ($g = \text{ones}(5, 5)$);
- 3 $R \leftarrow$ one of: $\lambda_{\min}$ (Shi–Tomasi); $\det H / \text{tr } H$ (Noble); $\det H - \alpha(\text{tr } H)^2$ (Harris);
- 4 $R(R < \theta) \leftarrow 0$; return max R per tile (non-max suppression);

with $\det H = \lambda_{\min} \lambda_{\max} = S_{xx} S_{yy} - S_{xy}^2$, $\text{tr } H = S_{xx} + S_{yy}$.

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
α (0.04–0.06)	Edge penalty in Harris R	Larger: rejects more edges; smaller: more permissive
Threshold θ	Minimum cornerness	Higher: fewer, stronger corners
Window g	Region summed into H	Larger: smoother, rotation-robust

Short explanation. A pixel on a uniform wall is untrackable. Harris reuses LK’s matrix $H = A^T A$ (the structure tensor) and its eigenvalues to classify pixels: two large = corner, one large = edge, both small = flat. The response R compresses this into one number (Harris’s form is cheapest, avoiding eigen-decomposition); thresholding + non-max suppression keeps strong, spread-out corners.

2.3 Horn–Schunck

Purpose. Estimate a *dense* flow field for every pixel by combining the brightness-constancy data term with a global smoothness prior.

When to use. Dense flow (warping, interpolation, magnification); coherent motion; small motion.

When to avoid. Large motion (use a pyramid); sharp motion boundaries (quadratic smoothness blurs them); brightness-constancy violations.

The algorithm.

Minimise over the whole image

$$(u, v) = \arg \min_{u, v} \iint [(I_x u + I_y v + I_t)^2 + \alpha(|\nabla u|^2 + |\nabla v|^2)] dx dy.$$

Euler–Lagrange gives $(I_x u + I_y v + I_t)I_x - \alpha(u_{xx} + u_{yy}) = 0$ (and the v analogue). Using the Laplacian kernel

$$L = \begin{pmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{pmatrix}:$$

$$\begin{pmatrix} I_x^2 + \alpha L & I_x I_y \\ I_x I_y & I_y^2 + \alpha L \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} I_x I_t \\ I_y I_t \end{pmatrix},$$

a sparse $2N \times 2N$ system (≈ 7 non-zeros/row), solved iteratively.

Algorithm 3: Horn–Schunck iterative solver

Input: I_1, I_2 ; weight α ; iterations K **Output:** Dense flow (u, v) 1 Compute I_x, I_y, I_t ; init $u, v \leftarrow 0$;2 **for** $k = 1$ **to** K **do**3 neighbourhood averages $\bar{u}, \bar{v}$;4 $u \leftarrow \bar{u} - I_x \frac{I_x \bar{u} + I_y \bar{v} + I_t}{\alpha + I_x^2 + I_y^2}; \quad v \leftarrow \bar{v} - I_y \frac{I_x \bar{u} + I_y \bar{v} + I_t}{\alpha + I_x^2 + I_y^2};$ 5 **end**

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
α (smoothness)	Data vs. prior (ML vs. MAP)	Larger: smoother, washes out real motion; smaller: choppy/noisy
K (iterations)	Solver convergence	More: closer to exact, costlier

Short explanation. LK works on a window; HS solves the whole image at once and cures the under-determined “one equation, two unknowns” problem with a smoothness prior: neighbouring pixels should have similar flow. α sets the balance (small = trust data / ML, large = trust smooth prior / MAP). The huge system is very sparse, so it solves fast, but the quadratic penalty blurs true motion boundaries.

3 Applications of Motion Estimation (Chapter 3)

Three uses of estimated motion — redistributing pixel information across time.

3.1 Super-Resolution (ML / MAP)

Purpose. Fuse several sub-pixel-shifted low-resolution frames into one high-resolution image X .

When to use. Multiple LR frames of the same scene with *varied* sub-pixel shifts (camera shake helps); need $\gtrsim r^2$ frames for upscale factor r .

When to avoid. Identical sub-pixel offsets (no new information); inaccurate motion F_i ; ML alone (ringing — add a prior).

The algorithm.

Forward model per frame: $Y_i = DHF_iX + v_i$ (decimate D , blur H , warp F_i). **ML:** $\min_X \sum_i \|DHF_iX - Y_i\|_2^2$, solved by gradient descent

$$X_{n+1} = X_n - \beta \sum_i F_i^T H^T D^T (DHF_iX_n - Y_i).$$

MAP (adds a Gibbs/smoothness prior $P(x) \propto e^{-A(x)}$, e.g. Laplacian $A(x) = \|Lx\|$):

$$X_{\text{map}} = \arg \min_X \|DHF X - Y\|_2^2 + \lambda A(x), \quad X_{n+1} = X_n - \beta \sum_i F_i^T H^T D^T (DHF_iX_n - Y_i) + \gamma L(X_n).$$

Algorithm 4: Iterative super-resolution (ML/MAP)

Input: LR frames $\{Y_i\}$; blur H , decimation D ; step β ; (MAP) γ

Output: HR image X

- 1 Estimate motions F_i ($F_1 = \text{identity}$); init X (e.g. upsampled Y_1);
 - 2 **repeat**
 - 3 residual per frame $DHF_iX - Y_i$; back-project by $F_i^T H^T D^T$;
 - 4 $X \leftarrow X - \beta \sum_i F_i^T H^T D^T (DHF_iX - Y_i)$ (MAP: $+\gamma L(X)$);
 - 5 **until** convergence;
-

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
β (step)	Gradient-descent rate	Larger: faster but may diverge
λ, γ (prior weight)	Smoothness vs. data (MAP)	Larger: smoother, less detail; smaller: sharper, noisier
# frames	Recoverable detail	Need $\gtrsim r^2$ with varied sub-pixel shifts

Short explanation. Each LR frame samples the HR scene on a slightly different sub-pixel grid, so combining them recovers detail. The forward model chains warp→blur→downsample; inverting it as least squares is a huge system, so it is solved by gradient descent using function calls (warp/blur/downsample), not giant matrices. ML alone rings, so MAP adds a smoothness prior — the same data-plus-regulariser recipe as HS.

3.2 BRIEF Descriptor

Purpose. Describe a keypoint with a fast binary “fingerprint” for matching.

When to use. Fast feature matching (e.g. inside hyperlapse); brightness-change resilience; matched by Hamming distance (XOR + popcount).

When to avoid. When rotation invariance is required (BRIEF is not rotation-invariant by default).

The algorithm.

For a set of fixed pixel-pair positions (p_i, q_i) in a window, output bit = 1 if $p_i > q_i$ else 0; concatenate into a binary string. Match two descriptors by Hamming distance.

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
# pixel pairs	Descriptor length	More bits: more distinctive, slower
Window size	Spatial support	Larger: more context, less localised
Pair pattern	Fixed sampling layout	Determines robustness/repeatability

Short explanation. BRIEF picks fixed pixel pairs around a corner and writes 1 if the first is brighter than the second, giving a bit string. Because it only compares relative brightness it resists lighting changes, and matching is a single XOR + popcount — extremely fast.

3.3 RANSAC

Purpose. Fit a model robustly when the data contains outliers (wrong matches).

When to use. Any fit with outliers: estimating a transform from feature matches, line/plane fitting, fundamental matrix. Always precede a transform-from-matches LS fit with it.

When to avoid. Very high outlier ratios make K explode; needs a sensible inlier threshold t .

The algorithm.

Repeat: sample the minimal #points (n : 2 for a line, 3 for affine), fit, count inliers (residual $< t$), keep the model with the most inliers; refit on all inliers. Number of draws:

$$K = \frac{\log(1 - P)}{\log(1 - W^n)}, \quad W = \text{inlier fraction}, \quad P = \text{target success prob.}$$

Algorithm 5: RANSAC

Input: Matches; model size n ; inlier threshold t ; support d ; draws K

Output: Robust model

```

1 for  $K$  times do
2   sample  $n$  points; compute model; count inliers ( $< t$ );
3   if #inliers  $> d$  then
4     keep as candidate
5   end
6 end
7 Recompute model using all inliers of the best candidate;
```

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
n (sample size)	Points to fit the model	Minimal n maximises all-inlier draw probability
t (inlier threshold)	Inlier/outlier cutoff	Larger: more inliers, looser fit; smaller: stricter
K (iterations)	Success probability	Set via the formula; higher P or lower $W \Rightarrow$ larger K

Short explanation. One bad match wrecks a least-squares fit. RANSAC samples the fewest points needed to define a model, counts how many other points agree, repeats many times, and keeps the model with the most votes, then refits on the inliers. Using the minimal sample keeps the “all-inliers” probability high, and the formula for K says how many tries guarantee success with probability P .

3.4 Hyperlapse (cost + dynamic programming)

Purpose. Turn a long jittery video into a smooth sped-up one by selecting frames that are smooth continuations of each other (not uniformly spaced).

When to use. Stabilised timelapse from hand-held / wearable footage; when uniform sub-sampling looks jittery.

When to avoid. When exact playback timing must be uniform (uniform sub-sampling then wins on timing).

The algorithm.

Match cost between frames i, j (feature points via Harris + BRIEF, transform $T(i, j)$ via affine + RANSAC): $C_r(i, j) = \frac{1}{n} \sum_p \|(x_p, y_p)_j^T - T(i, j)(x_p, y_p)_i^T\|_2$. Full cost adds a speedup term and a constant-pace term:

$$C(h, i, j, \mu) = C_m(i, j) + \lambda_s C_s(i, j, \mu) + \lambda_a C_a(h, i, j),$$

$C_s = \min(\|(j - i) - \mu\|_2^2, T_s)$, $C_a = \min(\|(j - i) - (i - h)\|_2^2, T_a)$. Minimise the total over paths by a 2-pass dynamic program (build cost matrix, back-trace optimal path).

Algorithm 6: Hyperlapse frame selection (DP)

Input: Frames; desired speedup v ; window w ; weights λ_s, λ_a

Output: Frame path P

- 1 Init: $D_v(i, j) = C_m(i, j) + \lambda_s C_s(i, j, v)$ for gaps $\leq w$;
 - 2 Pass I: $D(i, j) = C_m(i, j) + \lambda_s C_s + \min_k [D_v(i - k, i) + \lambda_a C_a(i - k, i, j)]$; store argmin in T ;
 - 3 Pass II: $P \leftarrow \text{Back-Trace}(T)$;
-

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
μ / v (speedup)	Target frames skipped	Larger: faster video
λ_s (speedup weight)	Adherence to target rate	Usually $\gg \lambda_a$ (timing matters less than smoothness)
λ_a (accel. weight)	Pace uniformity	Larger: more constant intervals
window w	Max jump considered	Larger: more options, costlier DP

Short explanation. Keeping every k -th frame gives violent jitter because samples land at random phases of the camera's bob. Instead, a cost measures how smoothly frame j follows frame i (via matched features and a fitted transform), plus penalties for missing the target speedup and for uneven pacing. A two-pass dynamic program finds the frame-selection path minimising the total cost.

4 Online Tracking: Kalman & Particle Filters (Chapter 4)

Online = frames arrive one at a time. A filter fuses the accumulated belief with each new measurement: *predict* + *correct*.

4.1 Bayesian Recursive Tracking (derivation)

Purpose. Derive the recursive posterior that underlies both Kalman and particle filters.

When to use. Any online tracking where a belief over the state history is updated recursively.

When to avoid. When the two assumptions (Markov state; observation depends only on current state) do not hold.

The algorithm.

With state x_t , measurements z_t : **Markov** $p(x_t | x_{1:t-1}) = p(x_t | x_{t-1})$ so $p(x_{1:t}) = p(x_1) \prod_{j \geq 2} p(x_j | x_{j-1})$; **conditional independence** $p(z_{1:t} | x_{1:t}) = \prod_{\tau} p(z_{\tau} | x_{\tau})$. Bayes then gives

$$p(x_{1:t} | z_{1:t}) \propto \underbrace{p(z_t | x_t)}_{\text{likelihood}} \underbrace{p(x_t | x_{t-1})}_{\text{motion model}} \underbrace{p(x_{1:t-1} | z_{1:t-1})}_{\text{recursion}},$$

and marginalising gives filtering $p(x_t | z_{1:t}) = C p(z_t | x_t) \int p(x_t | x_{t-1}) p(x_{t-1} | z_{1:t-1}) dx_{t-1}$.

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
$p(x_t x_{t-1})$	Motion / transition model	Constant position/velocity/acceleration
$p(z_t x_t)$	Measurement likelihood	e.g. $e^{-\text{SSD}}$ template score
prior (recursion)	Carried-forward belief	No need to store the video

Short explanation. Two assumptions — the future depends only on the present (Markov), and each observation depends only on the current state — collapse Bayes' rule into *likelihood* $\times$ *motion model* $\times$ *previous posterior*. That single recursion is what Kalman and particle filters implement.

4.2 Kalman Filter

Purpose. Optimally fuse a linear prediction with a noisy measurement under Gaussian noise.

When to use. Roughly linear dynamics, roughly Gaussian noise, unimodal belief; cheap ($O(n^3)$ in state dim).

When to avoid. Non-linear dynamics, non-Gaussian noise, or multi-modal beliefs (use a particle filter / EKF).

The algorithm.

Model $x_{k+1} = \Phi x_k + w_k$, $z_k = H x_k + v_k$, $Q = E[ww^T]$, $R = E[vv^T]$:

$$\begin{aligned} \text{Predict: } \hat{x}'_{k+1} &= \Phi \hat{x}_k, & P'_{k+1} &= \Phi P_k \Phi^T + Q, \\ \text{Gain: } K_k &= P'_k H^T (H P'_k H^T + R)^{-1}, \\ \text{Update: } \hat{x}_k &= \hat{x}'_k + K_k (z_k - H \hat{x}'_k), & P_k &= (I - K_k H) P'_k. \end{aligned}$$

Variable meanings:

- $\hat{x}_k$ = *state estimate at frame k* (after measurement update; this is your best guess right now)
- $\hat{x}'_k$ = *predicted state before measurement* (from the motion model Φ alone, uncertainty is high)
- z_k = *measurement at frame k* (e.g., LK optical flow, template match location; noisy data)
- P_k = *uncertainty (error covariance) of state estimate*; small = confident, large = uncertain
- P'_k = *uncertainty after prediction*; always grows because the motion model drifts
- K_k = *Kalman gain* (how much to trust the new measurement vs. the prediction; the key dial)
- Φ = *state transition matrix* (e.g., position $\rightarrow$ new position, or $[x, v] \rightarrow [x + v\Delta t, v]$)
- H = *measurement matrix* (extracts what you measured from the state; often identity)
- Q = *process noise covariance* (how much does the real world deviate from Φx_k ?)

- R = measurement noise covariance (how noisy is z_k ?)

K minimises $\text{tr}(P_k)$; in 1-D $K = P'/(P' + R)$ ($R \rightarrow 0 \Rightarrow K \rightarrow 1$ trust data; $R \rightarrow \infty \Rightarrow K \rightarrow 0$ trust model).

Algorithm 7: Kalman filter

Input: Φ, H, Q, R ; init $\hat{x}_0, P_0$; measurements z_k

Output: Filtered $\hat{x}_k$

```

1 for each frame  $k$  do
2   Predict:  $\hat{x}'_k = \Phi \hat{x}_{k-1}$ ,  $P'_k = \Phi P_{k-1} \Phi^T + Q$ ;
3   Gain:  $K_k = P'_k H^T (H P'_k H^T + R)^{-1}$ ;
4   Measure  $z_k$  (LK / template match near  $\hat{x}'_k$ );
5   Update:  $\hat{x}_k = \hat{x}'_k + K_k (z_k - H \hat{x}'_k)$ ,  $P_k = (I - K_k H) P'_k$ ;
6 end

```

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
Q (process noise)	Trust in motion model	Too small: over-confident, drifts in occlusion; too big: jumpy
R (measurement noise)	Trust in sensor	R small $\Rightarrow$ follow data; R large $\Rightarrow$ follow model
State vector	What is tracked	Add velocity/accel to coast through occlusions

Short explanation. Each frame the filter predicts (push the state through Φ ; uncertainty grows by Q) then updates (blend the measurement via gain K ; uncertainty shrinks). K is a trust dial set by R versus the predicted covariance. Optimal — but only under linear dynamics and Gaussian noise.

4.3 Particle Filter (Condensation)

Purpose. Track a non-Gaussian / multi-modal belief as a cloud of N weighted samples.

When to use. Non-linear dynamics, non-Gaussian noise, multiple hypotheses (look-alike targets); 100–200 particles.

When to avoid. When Kalman’s assumptions hold (cheaper); cost is $O(NM)$ per frame; beware sample impoverishment.

The algorithm.

Samples $\{S_t^{(n)}, \pi_t^{(n)}, C_t^{(n)}\}$ (S state, π weight, C cumulative weight) implement the recursion of §4.1.

Algorithm 8: Condensation (particle filter)

Input: $\{S_{t-1}^{(n)}, \pi_{t-1}^{(n)}, C_{t-1}^{(n)}\}_{n=1}^N$

Output: New particle set + estimate

```

1 for  $n = 1$  to  $N$  do
2   Select: draw  $r \sim U[0, 1]$ ; smallest  $j$  with  $C_{t-1}^{(j)} > r$ ;  $S_t'^{(n)} = S_{t-1}^{(j)}$ ;
3   Predict:  $S_t^{(n)} = A S_t'^{(n)} + B w_t^{(n)}$ ;                                     // drift + diffuse
4   Measure:  $\pi_t^{(n)} = p(z_t | x_t = S_t^{(n)})$ ;                               // e.g.  $e^{-\text{SSD}}$ 
5 end
6 Normalise  $\sum_n \pi_t^{(n)} = 1$ ;  $C_t^{(0)} = 0$ ,  $C_t^{(n)} = C_t^{(n-1)} + \pi_t^{(n)}$ ;
7 return  $\sum_n \pi_t^{(n)} f(S_t^{(n)})$  (or max-weight particle);

```

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
N (particles)	Approximation quality	More: better fidelity, costlier; too few: poor coverage
Diffusion noise	Exploration	Prevents impoverishment; too much: noisy estimate
Output rule	Max vs. weighted mean	Max: sharp but jittery; mean: stable but can fall between modes

Short explanation. A Gaussian cannot represent “on the table OR under the chair.” Particles store the belief as weighted samples of any shape. Each frame: resample (duplicate heavy particles via the cumulative-weight roulette), predict (motion + noise), re-weight by measurement similarity. Output is a weighted average (smooth) or the max-weight particle (sharp).

5 Dynamic Texture Synthesis (Chapter 5)

5.1 SVD + ARMA State-Space Texture Synthesis

Purpose. Learn a low-dimensional linear state-space model of a stochastic texture and drive it with noise to generate endless novel frames.

When to use. “Dynamic textures” — statistically stationary content that never repeats (fire, smoke, water, grass). State dim $n \sim 50$, image dim $m \sim 200 \times 300$.

When to avoid. Structured objects (a face, a person walking): the linear low-rank model produces blurry blobs — use GAN/VAE.

The algorithm.

Model $x(t+1) = Ax(t) + Bv(t)$, $y(t) = Cx(t) + w(t)$. Stack frames $Y_1^\tau = [y(1) \cdots y(\tau)]$; with $C^T C = I$ take the SVD $Y_1^\tau = U \Sigma V^T \Rightarrow \hat{C} = U$, $\hat{X} = \Sigma V^T$; fit dynamics by LS $\hat{A} = \arg \min_A \|X_1^\tau - A X_0^{\tau-1}\|$ (AR view: $x_t = C + \sum_i p_i x_{t-i} + \epsilon_t$).

Algorithm 9: Dynamic-texture learning + synthesis

Input: Frames $y(1..\tau)$; state dim n ; noise levels

Output: Endless synthetic frames

- 1 Vectorise frames into columns Y ; $U, \Sigma, V^T \leftarrow \text{SVD}(Y)$ truncated to n ;
 - 2 $\hat{C} \leftarrow U$; $\hat{X} \leftarrow \Sigma V^T$; $\hat{A} \leftarrow \arg \min_A \|X_1^\tau - A X_0^{\tau-1}\|$;
 - 3 pick start state x ;
 - 4 **while** *generating* **do**
 - 5 $x \leftarrow \hat{A}x + \epsilon$; output $y = \hat{C}x + w$;
 - 6 **end**
-

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
n (state dim)	Rank kept from SVD ($\geq 95\%$ variance)	Larger: sharper, costlier; smaller: blurrier
State noise v	Trajectory exploration	Keeps it non-deterministic; too big: unnatural
Image noise w	Natural look	Hides low-rank seams; too big: grainy
$C^T C = I$	Canonical solution	Removes the T -ambiguity ($A \rightarrow T A T^{-1}$)

Short explanation. A dynamic texture is a hidden low-dimensional state that drifts by a linear rule and renders to an image by another linear map — a Kalman model whose A, C are estimated from data (system identification). The SVD of stacked frames gives the patterns ($C = U$) and their time-mixing ($x_t = \Sigma V^T$); a least-squares fit gives A . Driving the state forward with noise generates infinitely many plausible frames.

6 Layer Representation (Chapter 6)

6.1 Layer Decomposition (LK + K-means) and Alpha Compositing

Purpose. Given a composited video and the number of layers k , recover each layer, its motion, and its mask by segmenting the motion into k affine motions.

When to use. Videos built from a few coherently-moving layers (background + a few objects); editing, object removal, background swap. Start with $k = 2-3$.

When to avoid. Non-coherent / highly deformable motion; wrong k (too small lumps motions, too large splits objects); poor K-means init (empty clusters).

The algorithm.

Alpha-compositing synthesis model $I_k = I_{k-1}(1 - \alpha_k) + E_k\alpha_k$ (layers E_k , masks α_k). Analysis is an EM-style alternation:

Algorithm 10: Layer decomposition (LK $\rightarrow$ K-means loop)

Input: Frame pair I_1, I_2 ; # layers k

Output: Per-pixel layer assignment, per-layer affine motion

```

1 repeat
2   | LK: fit a 6-D affine motion  $V \in \mathbb{R}^6$  on random windows;
3   | K-means: cluster the window motions in  $\mathbb{R}^6$  into  $k$  groups (layers);
4   | Refit LK per layer on its pixels;
5 until convergence;
```

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
k (# layers)	Number of motion groups	Too small: lumps motions; too large: splits objects (MDL picks k)
Motion model	Affine (6) vs. homography (8)	Affine usually enough; homography for planar perspective
# random windows	Sampling of motions	More: accurate, slower; random sub-sampling is standard

Short explanation. Movies are layered sandwiches (background, characters, effects) combined by alpha compositing $I = E\alpha + I_{\text{prev}}(1 - \alpha)$. To invert this, assume each layer moves by one affine transform (6 numbers): fit LK motions on windows, cluster them in $\mathbb{R}^6$ with K-means (each cluster = a layer), refit per layer, and iterate — the classic “assume A to estimate B, swap, repeat”.

6.2 K-means Clustering

Purpose. Partition points $x \in \mathbb{R}^d$ into k groups minimising total squared-error distortion.

When to use. Grouping vectors by similarity (motion vectors into layers; vector quantisation) with known k .

When to avoid. Unknown k (needs MDL/trial); non-convex/unequal clusters; poor init (local minima).

The algorithm.

Minimise Distortion = $\sum_i \|x_i - c_{\text{Encode}(x_i)}\|^2$; setting $\partial/\partial c_j = 0$ gives $c_j = \frac{1}{|s_j|} \sum_{i \in s_j} x_i$.

Algorithm 11: K-means (Lloyd)

Input: Points $\{x_i\}_{i=1}^n$; # clusters k

Output: Prototypes $c_1, \dots, c_k$

```

1 Initialise  $c_1, \dots, c_k$  (random /  $k$ -means++);
2 repeat
3   | Assign each  $x_i$  to nearest  $c_j$ ;
4   | Update  $c_j \leftarrow \frac{1}{|s_j|} \sum_{i \in s_j} x_i$ ;
5 until assignments stop changing;
```

Termination (guaranteed). Both steps never increase the distortion (nearest-centre assignment minimises it for fixed centres; the mean minimises it for fixed assignments), so distortion is monotonically non-increasing. Since there are only finitely many partitions ($\leq k^n$) and the cost cannot cycle upward, the algorithm stops after finitely many steps — at a local (not necessarily global) minimum.

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
k (# clusters)	Granularity	Larger k : lower distortion (0 at $k = n$); MDL picks optimal k
Initialisation	Which local minimum	k -means++ avoids empty clusters
Restarts	Robustness	Keep the lowest-distortion run

Short explanation. Drop k centres, assign each point to its nearest, move each centre to its members' mean, repeat. Each step is a distortion minimiser, so the cost only ever drops; with finitely many partitions and no upward cycling, K-means is guaranteed to terminate — possibly at a local minimum, which is why initialisation and restarts matter.

7 Video Magnification (Chapter 7)

7.1 Eulerian Video Magnification

Purpose. Amplify motions too small to see (a pulse, a swaying building) without estimating motion.

When to use. Sub-pixel, low-amplitude, dense changes; a steady (tripod) camera; known target frequency band.

When to avoid. Motion $\gg 1$ pixel (Taylor breaks $\Rightarrow$ ringing — use pyramid magnification); shaky camera (shake gets amplified too).

The algorithm.

For $I(x, t) = f(x + \delta(t))$, first-order Taylor gives $I(x, t) \approx f(x) + \underbrace{\delta(t) \partial f / \partial x}_{B(x, t)}$. Adding αB back yields

$$\tilde{I}(x, t) = I(x, t) + \alpha B(x, t) \approx f(x + (1 + \alpha)\delta(t)),$$

so motion is amplified by α — *without* estimating δ . In practice $B \approx I_2 - I_1$ (ideally band-passed in time first).

Algorithm 12: Eulerian magnification

Input: Video frames; amplification α ; (optional) temporal band-pass

Output: Motion-magnified video

```

1 foreach location (independently) do
2   |   temporal signal  $\rightarrow$  band-pass to the target frequency  $\rightarrow$  get  $B \approx I_2 - I_1$ ;
3   |   output  $\tilde{I} = I + \alpha B$ ;
4 end
```

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
α (amplification)	Motion gain (typ. 5–50)	Higher: reveals more but more ringing artifacts
Band-pass band	Frequencies amplified	Match target (heart ≈ 1 Hz); cuts noise
Pyramid	Handles > 1 px motion	Amplify each scale separately to avoid ringing

Short explanation. Lagrangian methods follow each pixel (need motion estimation); Eulerian methods sit at a fixed location and watch its *intensity profile* — the value of that one pixel over time — change. A tiny spatial shift δ changes the pixel value by $\delta \times \text{slope}$ (Taylor); multiply that temporal change by α and add it back and the pixel “moves” as if by $(1 + \alpha)\delta$ — amplification with no motion estimation. Band-pass first so only the target motion (not lighting drift/noise) is amplified.

8 Background Subtraction (Chapter 8)

Why GMM? (motivation from the intensity profile). Fix one pixel and plot its value across every frame of the video — its *intensity profile*. Ideally it sits at the background colour, jumps when a moving object passes over it, then returns. In practice a pixel often has *several* typical background values (e.g. a leaf blowing in front of sky), so its profile oscillates between a few clusters even with no foreground object present. The naive fix is to collect many frames, build a histogram of that pixel’s values, and cluster it (the big clusters = background modes, e.g. “leaf” and “sky”; a small outlier cluster = a passing object). This works, but has three problems: (1) *delay* — you must wait to collect enough frames before clustering; (2) *cost* — re-clustering per pixel is expensive; (3) *memory* — you must store the frame history. The **GMM** below fixes all three by replacing batch clustering with an *online, incremental* update: each new pixel value nudges an existing Gaussian or spawns a new one, with no stored history.

8.1 Per-pixel Gaussian Mixture Model (GMM)

Purpose. Classify each pixel as background or a moving object online, robust to repetitive background motion (rustling leaves).

When to use. Static-camera surveillance with multimodal backgrounds; low memory ($O(K)$ state per pixel).

When to avoid. Sudden lighting changes, camera shake (false positives until it adapts); slow objects get absorbed; shadows flagged as foreground.

The algorithm.

Each pixel’s value is modelled by K Gaussians $f_x(x | \Phi) = \sum_k \omega_k f_{x|k}(x | k, \theta_k)$, $\theta_k = (\mu_k, \sigma_k)$, $\sum_k \omega_k = 1$. Decision by MAP $\hat{k} = \arg \max_k \omega_k f_{x|k}$. Sort Gaussians by ω_k / σ_k (frequent + tight = background) and take the top ones with $B = \arg \min_b (\sum_{k \leq b} \omega_k / \sigma_k > T)$ as background. Online update per new value x_t :

$$\omega_{k,t} = (1 - \alpha)\omega_{k,t-1} + \alpha M_{k,t}, \quad \mu_t = (1 - \rho)\mu_{t-1} + \rho x_t, \quad \sigma_t^2 = (1 - \rho)\sigma_{t-1}^2 + \rho(x_t - \mu_t)^2,$$

where $M_{k,t} = 1$ if x_t matches Gaussian k (within 2.5σ), else 0, and $\rho = \alpha p(k | x_t) / \hat{\omega}_{k,t}$.

Algorithm 13: Online GMM background subtraction

Input: New pixel value x_t ; K Gaussians; rates α, ρ ; threshold T

Output: Background / foreground label

- 1 **if** x_t within 2.5σ of some Gaussian k **then**
 - 2 | nudge μ_k, σ_k toward x_t ; increase ω_k ;
 - 3 **else**
 - 4 | replace lowest-weight Gaussian with one centred at x_t (low ω , high σ);
 - 5 **end**
 - 6 Renormalise weights; sort by ω_k / σ_k ; background = top until $\sum > T$;
 - 7 Label x_t foreground iff its Gaussian is not background;
-

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
K (Gaussians)	Modes modelled (3–5)	Fewer: miss multimodal BG; more: fit noise
α (weight rate)	How fast new patterns become BG	≈ 0.01 ; too fast: parked car $\rightarrow$ BG; too slow: no adaptation
T (BG threshold)	BG fraction (0.7–0.9)	Higher: more count as BG, fewer false alarms
Match (2.5σ)	New-object sensitivity	Tight: detect changes; loose: absorb noise

Short explanation. Each pixel keeps its own mixture of a few Gaussians (mean colour, width, popularity). A new value that matches an existing Gaussian nudges it; otherwise it replaces the weakest one (likely a moving object). Background Gaussians are the frequent *and* tight ones (high ω/σ) whose weights sum past T ; everything else is foreground. It is fully online and memory-efficient.

9 Video Colorization & Geodesic Distance (Chapter 9)

Propagate a few colour scribbles to every pixel: fast within similar-luminance regions, slow across edges. The vehicle is *geodesic distance*, computed by Dijkstra or fast sweeping.

9.1 Geodesic Distance Colorization

Purpose. Fill the two chrominance channels of a B&W video from sparse colour scribbles, respecting luminance edges.

When to use. Region-wise-smooth colour (cartoon-like); colour edges coincide with luminance edges; video needs far fewer scribbles (time axis carries colour).

When to avoid. Saturated luminance with varying colour (fire); smooth colour gradients (sunset); specular highlights.

The algorithm.

Path length weighted by contour-crossing $L(p) = \int_p |\dot{p}(s), \nabla Y| dl$, distance $d_L(x, y) = \min_p L(p)$, distance map $D_k(x) = d_L(x, \Omega_k)$ to scribble k . Blend by a distance-decaying convex combination:

$$c(x) = \frac{\sum_k W(D_k(x)) c_k}{\sum_k W(D_k(x))}, \quad W(D) = \frac{1}{D^b}, \quad b \in [1, 6].$$

Algorithm 14: Geodesic colorization

Input: Luminance Y ; scribbles $\{\Omega_k, c_k\}$; exponent b

Output: Chrominance $c(x)$ for all pixels

```

1 foreach scribble  $k$  do
2   |  $D_k \leftarrow$  geodesic distance from  $\Omega_k$  (Dijkstra / fast sweeping);
3 end
4 foreach pixel  $x$  do
5   |  $c(x) \leftarrow \sum_k D_k(x)^{-b} c_k / \sum_k D_k(x)^{-b}$ ;
6 end
```

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
b in $1/D^b$ (1–6)	Transition sharpness	Larger: sharper boundaries; smaller: smoother blends
Edge weight $W_e = Y(p) - Y(q) $	How edges act as walls	Gradient magnitude blocks colour bleed
Scribble placement	Region coverage	Inside regions, not on edges; more in large regions
Video neighbourhood	4 (image) vs. 6 (+time)	Time edges let colour flow between frames

Short explanation. Redefine distance to follow content: staying inside one region costs almost nothing, crossing an edge costs a lot. For each scribble, compute this geodesic distance to every pixel; each pixel's colour is a weighted vote of all scribbles with weight $1/D^b$ (nearby scribbles dominate). In video, adding a temporal edge (6-neighbourhood) lets colour flow through time, so far fewer scribbles are needed.

9.2 Dijkstra's Algorithm

Purpose. Compute the shortest (geodesic) distance from a source set to every node of a weighted graph.

When to use. Exact single-source distance maps on a CPU with a priority queue; $O(N \log N)$.

When to avoid. Massively parallel / GPU settings (the priority queue is hard to parallelise — use fast sweeping).

The algorithm.

Edge weight $W_e = |Y(x, y) - Y(x', y')|$; path length $L_p = \sum_{e \in P} W_e$.

Algorithm 15: Dijkstra (geodesic distance)

Input: Grid (Ω, E) , weights W ; source set Ω_s
Output: $D(x) = d_L(x, \Omega_s)$

```

1  $D|_{\Omega_s} = 0, D|_{\text{rest}} = \infty, Q = \Omega;$ 
2 while  $Q \neq \emptyset$  do
3    $x = \arg \min_{x \in Q} D(x);$  // priority queue  $O(\log N)$ 
4   foreach  $y \in N_1(x) \cap Q$  do
5      $D(y) \leftarrow \min\{D(y), D(x) + W(x, y)\};$ 
6   end
7    $Q \leftarrow Q \setminus \{x\};$ 
8 end
```

Correctness rests on the *Bellman principle*: any sub-path of an optimal path is optimal.

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
Edge weights	Cost of crossing gradients	Gradient-magnitude weighting makes edges walls
Neighbourhood N_1	4 (image) / 6 (video)	Larger: more path options, costlier

Short explanation. Water spreads from the source pixels but flows slowly through high-gradient regions; the pixel that is “wet first” takes that distance. Pop the closest unvisited pixel, relax its neighbours, repeat — $O(N \log N)$ with a priority queue. The Bellman principle (sub-paths of optimal paths are optimal) is what makes the greedy relaxation correct.

9.3 Fast Sweeping (Danielsson)

Purpose. Compute the geodesic distance map with parallel-friendly raster sweeps instead of a priority queue.

When to use. GPU / cache-friendly settings; often faster than Dijkstra in practice.

When to avoid. Highly spiralling geodesics need many sweeps; distances are not Euclidean so > 4 sweeps may be required.

The algorithm.

Update each pixel from already-visited neighbours in raster order; 4 sweeps cover the 4 directions; repeat sweeps until the distance map stops changing (spirals need $\geq 4N$ sweeps for N turns). Parallelise by updating a whole column at a time.

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
# sweeps	Directions / turns covered	4 for simple paths; more for spirals; iterate to convergence
Neighbourhood layout	Parallelisation	Column-wise update enables parallel sweeps

Short explanation. Dijkstra’s priority queue is hard to parallelise. Fast sweeping instead runs simple raster scans, updating each pixel from its already-visited neighbours; four sweeps handle the four directions and more are added until nothing changes. It is cache- and GPU-friendly and usually faster than Dijkstra.

10 Video Matting (Chapter 10)

Extract a foreground and composite it onto a new background without colour bleed: trimap $\rightarrow$ opacity map $\rightarrow$ matting. Reuses Ch. 9's geodesic machinery with a foreground-probability cost field.

10.1 Trimap Estimation (KDE + geodesic)

Purpose. Label each pixel confidently-foreground (1), confidently-background (0), or unsure (ϕ).

When to use. Given FG/BG scribbles; when colour alone is ambiguous (overlapping FG/BG colours) so geodesic proximity breaks ties.

When to avoid. Poorly-placed scribbles (need one per distinct colour region); wrong band radius ρ .

The algorithm.

Estimate colour distributions by KDE (Parzen window) $f(c | F) \propto \sum_{x \in \Omega_F} K(c, c(x))$, $K \propto e^{-\|c - c'\|^2 / 2\sigma^2}$; Bayes gives $P(F | c) = f(c | F) / (f(c | F) + f(c | B))$ (uniform prior). Compute geodesic distances D_F, D_B using $W(x) = \nabla_x P(F | c(x))$ as the field; classify $V_F = \{D_F < D_B\}$; the unsure band is a radius- ρ ball around the boundary $\Delta = \{D_F = D_B\}$:

$$\alpha_T(x) = 1 \text{ on } V_F \setminus B_\rho(\Delta), \quad \alpha_T(x) = 0 \text{ on } V_B \setminus B_\rho(\Delta), \quad \alpha_T(x) = \phi \text{ on } B_\rho(\Delta).$$

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
ρ (band radius)	Width of unsure band (5–15 px)	Too small: misses soft hair; too big: too much work for stage 2
σ (KDE kernel)	Colour-model smoothness	Larger: smoother distribution
Scribbles	Colour-region coverage	One per distinct FG/BG colour region

Short explanation. Place a soft Gaussian “bell” on each scribble pixel’s colour (KDE) to get FG/BG colour distributions, convert to $P(F | c)$ by Bayes, then compute geodesic distances to FG and BG scribbles. Classify by whichever is closer, and mark a narrow band around the boundary as “unsure”. Geodesic proximity resolves cases where FG and BG share colours.

10.2 Opacity Map (trimap refinement)

Purpose. Assign each unsure pixel a continuous $\alpha \in [0, 1]$.

When to use. Inside the narrow uncertain band, where local colour statistics differ from the global model.

When to avoid. Outside the band (already confident 0/1).

The algorithm.

Re-estimate FG/BG distributions using only boundary pixels (nearly 1-D along the boundary curve length τ), then combine local probability with geodesic distance:

$$\alpha(x) = \frac{w_F(x)}{w_F(x) + w_B(x)}, \quad w_F(x) = \tilde{D}_F(x)^{-r} \tilde{P}_F(x), \quad r \in (0, 2].$$

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
$r \in (0, 2]$	Distance weighting	Larger: sharper reliance on proximity
Local KDE window	Boundary colour model	Re-estimated from boundary pixels (richer local data)

Short explanation. Near the boundary there are many more FG/BG samples than the sparse scribbles, so re-estimate the colour models locally. Combining the local FG probability with the geodesic distances gives each unsure pixel a soft opacity α between 0 and 1.

10.3 Matting (pure-foreground recovery)

Purpose. Composite onto a new background without dragging the old background’s colour into fuzzy pixels.

When to use. The uncertain boundary band (hair, fuzzy edges) where a pixel mixes FG and BG colour.

When to avoid. Confident FG/BG regions (a plain α blend is fine there).

The algorithm.

A boundary pixel is $c(x) = \alpha f(x) + (1 - \alpha)b(x)$ (one equation, two unknowns). In a small window assume f, b match confident FG/BG neighbours; pick the pair that best explains $c(x)$:

$$(x_F^*, x_B^*) = \arg \min_{x_F, x_B} \|\alpha(x)c(x_F) + (1 - \alpha(x))c(x_B) - c(x)\|^2, \quad J(x) = \alpha c(x_F^*) + (1 - \alpha)B(x).$$

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
Window size	Neighbour search for f, b	Larger: more candidates, costlier; small windows usually suffice
New background B	Composite target	Provided by the user

Short explanation. A naive blend $J = \alpha c + (1 - \alpha)B$ drags the old background’s colour onto the new one at fuzzy pixels. Instead recover the *pure* foreground colour by finding, in a small window, the confident FG/BG pixel pair that best reproduces the mixed pixel, and composite that pure colour — avoiding colour bleed.

10.4 Fast KDE via Low-Rank Kernel

Purpose. Evaluate $f(x) = \sum_y k(x, y)w(y)$ (KDE) in linear rather than quadratic time.

When to use. Large images where naive KDE is $O(|\Omega| |\Omega'|)$; kernels with fast-decaying singular values (e.g. Gaussian).

When to avoid. Genuinely full-rank kernels (no speedup).

The algorithm.

If $K = \sum_{i=1}^p \phi_i \psi_i^T$ (rank $p \ll m, n$), then with $a_i = \sum_y \psi_i(y)w(y)$, $f(x) = \sum_i a_i \phi_i(x)$, total $O((m+n)p)$ instead of $O(mn)$ (the Fast Multipole Method factorises such kernels).

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
p (effective rank)	Approximation vs. speed	Smaller p : faster, less accurate
Kernel choice	Rank / decay	Pick low-rank (fast-decaying singular values)

Short explanation. Naive KDE sums over every scribble pixel at every output pixel — quadratic. If the kernel is (approximately) low rank, the double sum factors into two cheap passes, giving $O((m+n)p)$. Gaussians are formally infinite-rank but their singular values decay, so a small effective rank works.

11 Re-targeting & PatchMatch (Chapter 11)

Resize/edit an image while preserving a natural look, formalised by two losses (completeness + coherence) and solved fast by PatchMatch.

11.1 Bidirectional-Similarity Re-targeting

Purpose. Produce a target of new size/aspect that keeps all important source content (completeness) and invents nothing (coherence).

When to use. Content-aware resize, inpainting, reshuffling, object removal/preservation; images and video (3-D patches).

When to avoid. Bad initialisation (start from black $\Rightarrow$ garbage — use a hierarchical pyramid); straight lines/objects may bend (add constraints).

The algorithm.

With patch distance $d^2(P, Q) = \|P - Q\|_2^2$:

$$D_{\text{com}}(S, T) = \frac{1}{N_S} \sum_{P \in S} \min_{Q \in T} d^2(Q, P), \quad D_{\text{coh}}(S, T) = \frac{1}{N_T} \sum_{Q \in T} \min_{P \in S} d^2(P, Q),$$

$D = \alpha D_{\text{com}} + (1 - \alpha) D_{\text{coh}}$. Per-pixel optimum is a weighted average of voting source colours (set derivative to 0). Weighted constraints multiply patches by W_P (high = preserve, ≈ 0 = remove), keeping D convex.

Algorithm 16: Re-targeting by bidirectional similarity

Input: Source S ; target size Ω' ; balance α

Output: Target T

- 1 Initialise T^1 by hierarchical resize (gradual pyramid), *not* black;
 - 2 **repeat**
 - 3 reset $T^{k+1} \leftarrow 0$, weight map $C \leftarrow 0$;
 - 4 **Coherence:** each $Q \in T^k$ finds NN $P \in S$; add $\frac{1-\alpha}{N_T} P$ onto Q ; $C += \frac{1-\alpha}{N_T}$;
 - 5 **Completeness:** each $P' \in S$ finds NN $Q' \in T$; add $\frac{\alpha}{N_S} P'$ onto Q' ; $C += \frac{\alpha}{N_S}$;
 - 6 $T^{k+1} \leftarrow T^{k+1} / C$ (element-wise);
 - 7 **until** *convergence*;
-

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
α (0–1)	Completeness vs. coherence	Higher: keep more content; lower: smoother but riskier (start 0.5)
Patch size (5–7)	Context vs. sharpness	Larger: more context, blurrier
Constraint weights W_P	Preserve / remove	High: protect a face/line; ≈ 0 : erase an object
Pyramid step ($\sim 1.1\times$)	Init quality	Gradual resize avoids bad local minima

Short explanation. Two symmetric principles: every important source patch should appear in the target (completeness), and every target patch should exist in the source (coherence). Their weighted sum is minimised by setting each output pixel to a weighted average of the source patches voting for it. Good initialisation (a gradual resize pyramid) is essential, and the nearest-neighbour search is the bottleneck that PatchMatch accelerates.

11.2 PatchMatch

Purpose. Compute an approximate nearest-neighbour offset field in near real time.

When to use. Whenever brute-force $O(N^2)$ patch matching is too slow and matches are spatially coherent (most natural images).

When to avoid. Non-coherent correspondences; when an exact global NN is required.

The algorithm.

Find $f(x) = \arg \min_y \|P_x - Q_{x+f}\|_2^2$ using *propagation* (exploit coherence) + *random search* (escape local minima).

Algorithm 17: PatchMatch (1-NN)

Input: Source S , target T ; window W ; iterations K
Output: NN offset field f

```

1 Init  $f \sim U[-W/2, W/2]^2$ ;  $d(x) = \|P_x - Q_{x+f(x)}\|_2^2$ ;
2 for  $k = 1$  to  $K$  (alternating raster order) do
3   foreach  $x$  do
4     Propagation:  $f(x) \leftarrow \arg \min_{f \in \{f(x), f(x-e_1), f(x-e_2)\}} \|P_x - Q_{x+f}\|$ ;
5     Random search: test  $f(x) + r$  with shrinking radius; keep if it lowers  $d(x)$ ;
6   end
7 end
```

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
Patch size (5–7)	Context vs. sharpness	Larger: more context, blurrier
Iterations K (4–6)	NN-field convergence	More: better matches, costlier
Window W	Random-search radius	Shrinks each inner step

Short explanation. If patch (100,100) matches (200,50), its neighbour almost certainly matches with the same offset — so neighbours borrow each other’s offset instead of searching. PatchMatch alternates propagation (spread good offsets to causal neighbours, flipping scan direction each pass) with shrinking-radius random search. Together they converge in ~ 4 –6 passes, hundreds of times faster than brute force.

11.3 Applications: Inpainting, Reshuffling, Object Removal

Purpose. Fill missing pixels, move objects, or delete content — all as the same re-targeting optimisation with constraints, solved by PatchMatch.

When to use. Scratch/hole filling; rearranging objects; content-aware removal; mixing content from several sources.

When to avoid. Large semantically-structured holes (patch copying can’t hallucinate unseen structure).

The algorithm.

Inpainting: minimise $D(S, T)$ over the missing region Ω_m with $T|_{\Omega \setminus \Omega_m} = S|_{\Omega \setminus \Omega_m}$. *Reshuffling:* fix pasted-object pixels, fill the rest. *Removal:* set the object’s patch weights ≈ 0 so it cannot influence T ; *preservation:* weight protected patches (and their NN) high.

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
Mask Ω_m	Region to synthesise	Generous mask (+few px) avoids edge halos
Weights W_P	Preserve vs. remove	High: keep; ≈ 0 : remove
Source set	Where fill comes from	Multiple sources allowed

Short explanation. All these tasks reduce to “define which pixels are fixed and which are free, then minimise completeness + coherence with PatchMatch”. Weighting patches steers the result — high weight preserves a face or line, near-zero weight makes an object vanish as neighbours fill its space.

12 Intrinsic Images from Sequences (Chapter 12)

12.1 Weiss Intrinsic Images (median of derivative filters)

Purpose. From T images of a fixed scene under changing light, recover one reflectance (paint) image and T illumination (light) images.

When to use. A stationary camera/scene with only lighting varying (a day-long webcam); a handful of frames already suffices.

When to avoid. A single image (ill-posed: T equations, $T+1$ unknowns); non-sparse lighting; strongly non-linear camera response (assumed linear).

The algorithm.

In the log domain $i(x, y, t) = r(x, y) + l(x, y, t)$. Assume derivative-filter outputs of the lighting are sparse (Laplacian $P(x) \propto e^{-\alpha|x|}$). With filters $\{f_n\}$, $o_n = i \star f_n$, the ML estimate of the filtered reflectance is the *temporal median*

$$\hat{r}_n(x, y) = \text{median}_t o_n(x, y, t)$$

(because maximising the Laplacian likelihood minimises the ℓ_1 error, whose minimiser is the median). Reconstruct r from its filtered versions by the pseudo-inverse

$$\hat{r} = g \star \left(\sum_n f_n^r \star \hat{r}_n \right), \quad g \star \left(\sum_n f_n^r \star f_n \right) = \delta,$$

where $f_n^r(x, y) = f_n(-x, -y)$ and g is a precomputable, image-independent deconvolution filter. Then $\hat{l}(x, y, t) = i(x, y, t) - \hat{r}(x, y)$.

Algorithm 18: Weiss intrinsic-image estimation

Input: Log frames $i(\cdot, \cdot, t)$, $t = 1..T$; derivative filters $\{f_n\}$ (usually ∂_x, ∂_y)

Output: Reflectance $\hat{r}$; illuminations $\hat{l}(\cdot, \cdot, t)$

1 **foreach** filter f_n **do**

2 | $o_n(\cdot, \cdot, t) = i(\cdot, \cdot, t) \star f_n$; $\hat{r}_n = \text{median}_t o_n$;

3 **end**

4 $\hat{r} \leftarrow g \star (\sum_n f_n^r \star \hat{r}_n)$;

// g precomputed from the filters

5 $\hat{l}(\cdot, \cdot, t) \leftarrow i(\cdot, \cdot, t) - \hat{r}$;

Convergence (Claim 2). If $p_\epsilon = P(|f_n \star l| < \epsilon)$ then $P(|\hat{r}_n - r_n| < \epsilon) \geq \sum_{k=0}^{\lfloor T/2 \rfloor} \binom{T}{k} (1 - p_\epsilon)^{T-k} p_\epsilon^k$; sparser lighting (larger p_ϵ) converges faster — $p_\epsilon = 0.85$ gives ≥ 0.93 at $T = 3$ and ≥ 0.97 at $T = 5$.

Parameters and knobs.

Parameter	What it controls	Effect of increasing / decreasing
Filters $\{f_n\}$	Which gradients (usually ∂_x, ∂_y)	Two derivative filters are enough; g depends only on them
# frames T	Convergence certainty	More frames / sparser light $\Rightarrow$ higher accuracy
g	Reconstruction deconvolution	Precompute once; reuse on every sequence

Short explanation. Take logs so image = paint + light. The paint is constant across frames; the light's edges are sparse (mostly zero). So at each pixel, the *temporal median of the edge-filtered frames* throws away the rare shadow-edge outliers and keeps the paint's gradients; reconstruct an image from those gradients (a fixed deconvolution) to get reflectance, then subtract to get each frame's lighting. Crucially it is the median of *gradients*, not raw pixels — a permanently-shadowed pixel produces no edge and cancels, whereas a raw-pixel median (or min/mean) would bake the shadow into the paint.

Course-wide Pattern

Course-wide pattern. Almost every method reduces to: *define a cost, minimise it iteratively, add a regulariser*. Least squares handles over-determined systems (motion models, LK/HS, super-resolution, texture A); a smoothness/MAP prior resolves ambiguity (HS, super-resolution, colorization); dynamic programming gives shortest paths (geodesic/Dijkstra, hyperlapse); predict/update or propagation/randomisation drive the iterative solvers (Kalman, particle filter, PatchMatch, K-means); and low-rank/SVD structure recurs (texture synthesis, fast KDE, intrinsic-image reconstruction).

Parameter Index and Glossary

Quick reference: every parameter mentioned in this document, organised by algorithm. Use this to quickly find what a parameter does or how changing it affects the method.

Affine Transformation (2×3)

A (4 params) — Rotation/scale/shear

$t = (t_x, t_y)$ — Translation

point pairs — Determinacy of the fit

Applications: Inpainting, Reshuffling, Object Removal

Mask Ω_m — Region to synthesise

Weights W_P — Preserve vs. remove (≈ 0 : erase; high: protect)

Source set — Where fill comes from (multiple sources allowed)

BRIEF Descriptor

pixel pairs — Descriptor length (more: more distinctive, slower)

Window size — Spatial support (larger: more context, blurrier)

Pair pattern — Fixed sampling layout

Bayesian Recursive Tracking (derivation)

$p(x_t | x_{t-1})$ — Motion / transition model

$p(z_t | x_t)$ — Measurement likelihood

prior (recursion) — Carried-forward belief

Bidirectional-Similarity Re-targeting

α (0–1) — Completeness vs. coherence (higher: keep more content; lower: smoother)

Patch size (5–7) — Context vs. sharpness

Constraint weights W_P — Preserve / remove (high: protect a face/line; ≈ 0 : erase)

Pyramid step ($\sim 1.1\times$) — Gradual resize for good initialisation

Dijkstra’s Algorithm

Edge weights — Cost of crossing gradients (gradient magnitude)

Neighbourhood N_1 — 4-conn (image) or 6-conn (video with time)

Dynamic Texture Synthesis

n (state dim) — Rank kept from SVD ($\geq 95\%$ variance)

State noise v — Trajectory exploration (prevents determinism)

Image noise w — Natural look (hides low-rank seams)

Eulerian Video Magnification

α (amplification) — Motion gain, typically 5–50

Band-pass band — Target frequency range (match your signal)

Pyramid — Handles motions > 1 pixel

Fast KDE via Low-Rank Kernel

p (**effective rank**) — Approximation quality vs. speed

Kernel choice — Pick low-rank kernels (fast-decaying singular values)

Fast Sweeping (Danielsson)

sweeps — Directions / turns covered (4 for simple paths; more for spirals)

Neighbourhood layout — Enables parallel column-wise updates

Geodesic Distance Colorization

$b \in [1, 6]$ (**in** $1/D^b$) — Transition sharpness (larger: sharper boundaries)

Edge weight $|Y(p) - Y(q)|$ — How edges act as walls

Scribble placement — One per distinct FG/BG colour region (inside regions, not on edges)

Video neighbourhood — 4 (image only) vs. 6 (+ time; lets colour flow through frames)

GMM Background Subtraction

K (**Gaussians**) — Modes modelled, typically 3–5

α (**weight rate**) — How fast new patterns become background (≈ 0.01)

T (**BG threshold**) — Background fraction, 0.7–0.9

Match distance (2.5σ) — Detection sensitivity to new objects

Harris Corner Detector

α (**0.04–0.06**) — Edge penalty in Harris response R

Threshold θ — Minimum cornerness (higher: fewer, stronger corners)

Window g — Region summed into H (larger: smoother, rotation-robust)

Homography (3×3)

H (**9 entries, 8 free**) — Perspective warp parameters

point pairs — Determinacy (need ≥ 4 pairs)

Normalisation — Numerical stability (constrains overall scale)

Horn–Schunck

α (**smoothness**) — Data vs. prior; ML (small) vs. MAP (large)

K (**iterations**) — Solver convergence (more: closer to exact)

Hyperlapse

μ / v (**speedup**) — Target frames skipped (larger: faster video)

λ_s (**speedup weight**) — Adherence to target rate (usually $\gg \lambda_a$)

λ_a (**accel. weight**) — Pace uniformity (larger: more constant intervals)

window w — Max jump considered in DP (larger: more options)

Kalman Filter

Q (**process noise**) — Trust in motion model (too small: over-confident; too big: jumpy)

R (**measurement noise**) — Trust in sensor (R small: follow data; large: follow model)

State vector — What is tracked (add velocity/accel for occlusion coasting)

K-means Clustering

k (# clusters) — Granularity (larger k : lower distortion but more noise)

Initialisation — Which local minimum (k-means++ avoids empty clusters)

Restarts — Robustness (keep the lowest-distortion run)

Layer Decomposition

k (# layers) — Number of motion groups (too small: lumps motions; too large: splits objects)

Motion model — Affine (6) vs. homography (8)

random windows — Sampling of motions (more: accurate, slower)

Linear Transformation (2×2)

a, b, c, d — Rotation/scale/shear jointly (4 free parameters)

point pairs — Determinacy of the fit (need ≥ 2 pairs)

Lucas-Kanade

Window size n — # equations per solve (larger: robust but mixes motions; smaller: worse aperture)

Warp model — Translation (2) vs. affine (6)

Pyramid levels — Max recoverable motion (more levels: handle bigger motion)

Matting (pure-foreground recovery)

Window size — Neighbour search for foreground/background (small windows usually suffice)

New background B — Composite target (provided by user)

Opacity Map

$r \in (0, 2]$ — Distance weighting in opacity calculation

Local KDE window — Boundary colour model (re-estimated from boundary pixels)

Opacity Map (Trimap Refinement)

ρ (band radius) — Width of unsure band, 5–15 px

σ (KDE kernel) — Colour-model smoothness (larger: smoother distribution)

Scribbles — Colour-region coverage (one per distinct FG/BG colour)

Particle Filter (Condensation)

N (particles) — Approximation quality (more: better fidelity, costlier)

Diffusion noise — Exploration; prevents sample impoverishment

Output rule — Max vs. weighted mean (max: sharp but jittery; mean: stable)

PatchMatch

Patch size (5–7) — Context vs. sharpness

K iterations (4–6) — NN-field convergence

Window W — Random-search radius (shrinks each pass)

Projection Matrix (pinhole camera)

R, t (**extrinsics**) — Camera pose (6 params: 3 rotation, 3 translation)

f, k_u, k_v, u_0, v_0 (**intrinsics**) — Lens + sensor mapping (5 params, calibrated once)

Depth Z — Projects inversely to size ($1/Z$ shrinks distant objects)

RANSAC

n (**sample size**) — Points to fit the model (minimal n maximises all-inlier probability)

t (**inlier threshold**) — Inlier/outlier cutoff

K (**iterations**) — Success probability (set via the closed-form formula)

Re-targeting by Bidirectional Similarity

Initialisation T^1 — Start from hierarchical pyramid, not black

α (**0–1**) — Completeness vs. coherence (0.5 is a good start)

Super-Resolution (ML / MAP)

β (**step**) — Gradient-descent rate

λ, γ (**prior weights**) — Smoothness vs. data in MAP

frames — Recoverable detail (need $\gtrsim r^2$ with varied sub-pixel shifts)

Weiss Intrinsic Images

Filters $\{f_n\}$ — Which gradients (usually ∂_x, ∂_y)

frames T — Convergence certainty ($p_\epsilon = 0.85$: ≥ 0.93 at $T = 3$)

g — Reconstruction deconvolution (precomputed from filters; reuse on every sequence)